Making MOSS-TTS-Realtime Stream on an 8 GB Consumer GPU: Shape and Compilation Hygiene for Autoregressive Speech Synthesis

Hongyun Du
Independent Researcher
hdu1@my.nl.edu

November 1, 2026

Abstract

Autoregressive text-to-speech models are usually deployed on server GPUs, where the audio codec and the acoustic model can share one device. On an 8 GB consumer card they cannot: the 1.7B talker of MOSS-TTS-Realtime occupies 4.34 GiB and its codec 6.6 GB, so the codec must run on the CPU. Under that arrangement the model measured RTF 2.08 (synthesis seconds per audio second), which cannot sustain streaming. We report six conditions that together bring the same utterance to RTF 0.90 with 0.92 s to first audio on a GeForce RTX 3060 Ti. Four of the six concern tensor shapes, not computation: a growing KV cache, a growing attention mask, and an integer argument that made the compiler re-specialise on every frame. The remaining two are a missing Triton installation that had silently disabled compilation, and a decoder chunk size whose per-call cost dominated decoding. Quantising the talker to int8 reduces weights to 2.82 GiB at the same steady-state speed, at the cost of a much longer first compilation. We describe the measurements, the wrong turns that preceded them, and what the numbers do not establish.

1 Introduction

MOSS-TTS-Realtime is a 1.7B streaming text-to-speech model released under Apache-2.0. Its intended deployment is a server GPU large enough to hold both the talker and the 6.6 GB audio tokenizer, which is the arrangement reported in the model card and the one used by inference stacks such as vLLM-Omni.

Consumer hardware changes the problem. On an 8 GB card the talker alone leaves roughly 3 GiB, so the codec is pushed to the CPU. Streaming then requires the pipeline to produce audio faster than it plays back – RTF below 1.0 – with one device doing the acoustic modelling and the CPU doing waveform reconstruction.

We began from the model's own streaming harness and measured RTF 2.08 on a single sentence, with 1.02 s to first audio. This note reports what the time was actually spent on, and the six conditions that brought the same utterance to RTF 0.90. Four of them are not optimisations in the usual sense: they are removals of shape changes that made `torch.compile` recompile instead of reuse.

2 The system under study

MOSS-TTS-Realtime is a streaming text-to-speech model [1] built on the MOSS audio tokenizer [2]. Audio is represented as 16 codebooks at 12.5 frames per second. The model has three parts that matter here:

- a 1.7B talker, which advances the frame loop and decides the first codebook token;
- a depth transformer, which produces the remaining fifteen codebook tokens for each frame with one forward pass per codebook;
- an audio tokenizer (*codec*) that reconstructs waveform from the codebooks.

The talker and the depth transformer together occupy 4.34 GiB of bf16 weights, and the codec 6.6 GB. On an 8 GB card those do not fit together, which is what shapes the rest of this note.

3 Where the time goes

Before changing anything we instrumented a single frame, at 12.5 audio frames per second:

Component	Cost per frame
Talker, 1.7B forward	31 ms
Depth transformer, 16 codebook forwards	96 ms

The talker is close to its memory-bandwidth floor: 3.4 GB of bf16 weights read at roughly 448 GB/s is 7.6 ms, and the remainder is KV attention and kernel launch. The depth transformer, which decides sixteen codebook tokens per frame, cost three times as much. This is the opposite of the intuitive guess – the large model is not the expensive one – and it determined where compilation was worth applying.

Decoding was measured separately. Decoding a 6.5 s utterance in one call takes 0.49 s on CPU and 0.46 s on this GPU, a ratio of 1.1. The codec is not the bottleneck; the chunked decoding schedule was.

4 Six conditions

1. Triton must be installed. Without a working Triton, inductor cannot run and `torch.compile` silently falls back to eager. On Windows this is the `triton-windows` package. After installing it, PyTorch's own availability check reports Triton as present, and every earlier "compiled" measurement turned out to have been eager: the compiler had been silently declining to run.

2. Disable integer specialisation. `step()` passes `gen_step`, an integer that increments every frame. Dynamo specialises on integer arguments by default, so each frame created a new specialisation and triggered a recompile. With full compilation the model ran *13 times slower* than eager. Setting `torch._dynamo.config.specialize_int = False` reduced a whole utterance to two graphs.

3. Compile the depth transformer, not the talker. Given the profile above, compiling the 96 ms component is worth doing and compiling the 31 ms one is not; the latter also costs about 100 s of warmup. The depth transformer already carries a `torch.compile` decorator, so the correct action is to leave it enabled and to keep the talker out of the compiled path.

4. Pin the KV cache. A `StaticCache` allocated once, with an explicit `cache_position`, stops the cache from growing and therefore stops the shapes from changing. Two details cost time to discover: the configuration must be `model.config.language_config` (the top-level config has no `num_hidden_layers`), and the cache must be attached *before* prefill, or the model replaces it with a `DynamicCache`.

5. Fix the attention-mask length. `step()` grows its mask with `torch.cat`, which changes its length every frame and defeats the previous condition. Preallocating a mask of `max_cache_len` and writing the new position in place keeps the shape constant. The length should be modest: a 4096-entry mask costs more attention work per frame than a 512-entry one and returns nothing.

6. Decode in larger chunks. The streaming decoder's cost is dominated by per-call overhead rather than by token count, so a small chunk size is actively harmful (Table 1).

`chunk_frames`	Decode time	First audio	RTF
3 (default)	4.50 s	0.45 s	1.43
12	1.52 s	**0.92 s**	**0.90**
15	1.25 s	1.05 s	0.86
20	0.94 s	1.30 s	0.83
40	0.63 s	2.40 s	0.76

Table 1: Decode chunk size trades time-to-first-audio against RTF. We chose 12: first audio under a second while remaining below 1.0.

5 Measurements

Table 2 reports the same sentence before and after, and the effect of int8 weights. The int8 result contradicts the usual expectation that quantisation buys memory at the cost of speed: once the compile conditions apply, steady-state cost is unchanged, so the saving is purely in memory.

Configuration	First audio	RTF	Steady step	Talker memory
Eager, as shipped	1.02 s	2.08	109.4 ms	4.34 GiB
Six conditions, bf16	0.92 s	0.90	61.5 ms	4.34 GiB
Six conditions, int8	0.94–0.98	0.94–0.98	50.6 ms	**2.82 GiB**

Table 2: Single sentence, RTX 3060 Ti, codec on CPU. Third digit is noise: these are single runs.

Quantisation has its own cost, and it is not the quantisation itself. Converting weights takes 0.2 s; compiling the quantised graph took 818 s cold and about 300 s with a warm inductor cache, against 42 s for bf16. A system that reports readiness after loading weights will therefore make its first user wait several minutes. We gate readiness on a completed synthesis instead.

That gate has a subtlety worth stating: compilation is keyed by shape, and shape follows text length. Warming with a five-character sentence left the first real request paying 77 s. Warming over short, typical and long sentences brought the first real request to 7.03 s.

For context, the same model on a rented 24 GB A10 with the codec on the GPU and none of the above applied measured 698 ms to first audio, 8.76 s of generation for 6.56 s of audio, and RTF 1.59. A larger card alone did not cross 1.0.

5.1 What int8 costs in quality

Listening to one clip is not evidence, and the obvious comparison does not exist: at a fixed seed the two precisions produce different performances rather than an original and a distorted copy. Waveform correlation between bf16 and int8 was 0.016, and greedy decoding did not change it, because a small numeric difference flips one codebook token and the autoregressive path never returns to the bf16 trajectory. "int8 reproduces bf16" is therefore not a testable property.

What is testable is whether each precision, scored on its own over a set of utterances, is of comparable quality. We rendered 20 Chinese sentences covering statements, questions, exclamations and both short and long lines, once per precision, at temperature 0.8 and a fixed seed, and scored three standard measures:

Weights	CER (%)	SECS	UTMOS	mean duration
bf16	1.23	0.670	3.21	2.54 s
int8	0.56	0.672	3.29	2.55 s
difference	−0.67	+0.002	+0.077	+0.01 s

CER comes from a Paraformer pass with punctuation removed, SECS from ECAPA-TDNN embeddings against the reference clip, and UTMOS from SpeechMOS. Thresholds were fixed before the run: no worse than +1.0 point CER, −0.01 SECS, −0.05 UTMOS. All three land inside them, and all three lean marginally toward int8, which at $n = 20$ and a single seed is noise rather than an improvement. The defensible conclusion is that int8 is not measurably worse on intelligibility, speaker identity or predicted naturalness; it is not that it is better.

The metric that would settle prosody is a listening test, which we did not run. CER cannot see a flattened delivery, and UTMOS is a predictor rather than a listener.

6 Negative results

Four conclusions we held, and had to abandon:

- *"Windows cannot compile."* It can; Triton was simply absent.
- *"Compilation gives a 2.3× speedup."* True on a micro-benchmark whose shapes had stabilised during warmup, and false end to end, where recompilation dominated.
- *"Compile the whole model."* This produced a 13× slowdown and, in one configuration, an 818 s first utterance.
- *"Chunk size does not matter."* This was an artefact of a measurement bug: the benchmark swapped a variable the measurement path did not use. Once the decoder was wired correctly, chunk size turned out to be the single largest remaining lever.

We record these because each was reached by a plausible measurement, and because the last one shows that an instrument can be wrong in a way that looks like a result.

7 What this does not establish

The numbers come from one machine, one model revision, one prompt set, and single runs per configuration. We make no statistical claim, and the third digit should be read as noise.

We do not survey concurrent work on this model. These measurements were made independently, and any optimisation published elsewhere for MOSS-TTS-Realtime is neither evaluated nor compared against here.

The int8 comparison holds for 20 sentences, one seed and one voice. That is enough to catch a real regression and not enough to certify equality, and the duration column moves by 0.01 s where 0.1 s might be expected between two runs of the same configuration.

This note is an engineering report on one implementation, and it does not attempt a survey of concurrent work. The model itself is described in [1, 2]; the general techniques relied on here are `torch.compile`/Inductor [3] and Hugging Face's cache strategies [4].

We claim no new kernel and no new algorithm. What is reported is a set of conditions under which an existing implementation reaches RTF below one on one consumer card, together with the measurements that show which condition matters and which does not.

8 Reproduction

A self-contained script accompanies this note. It takes the two model repositories as input, installs the six conditions, and prints first-audio latency, RTF, steady per-step cost, and the generation/decode split. Its output on the machine above, run from a clean directory:

```
pass 1: first 19726 ms  RTF 4.12  step 46.5 ms  decode 1.54 s
pass 2: first   939 ms  RTF 0.93  step 46.9 ms  decode 1.58 s
```

The first pass pays compilation; the second is the steady state. The split is also the diagnostic: a large `decode` figure points at condition six, a large `step` figure at conditions two to five.

References

[1] Y. Gong et al. *MOSS-TTS Technical Report*. arXiv:2603.18090, 2026.

[2] Y. Gong et al. *MOSS-Audio-Tokenizer: Scaling Audio Tokenizers for Future Audio Foundation Models*. arXiv:2602.10934, 2026.

[3] PyTorch Team. *torch.compile* and *torch._dynamo* documentation.

[4] Hugging Face Transformers. *Cache strategies: StaticCache and cache_position*.